

BackTracking vs Recursion

Feature	Recursion	Backtracking
Definition	Function calling itself	Technique to explore all possibilities
Choices	Usually one	Multiple choices
Undo step	Not required	Required
Purpose	Solve smaller subproblems	Generate / search all valid solutions
Example	factorial, tree traversal	N-Queens, Sudoku, permutations

Graph

Graph is complex type of ds

V vertices and E Edges

$V_1 - e \rightarrow V_2$

types of graph

directed undirected- there will be direction

cyclinc Acyclic -> there will be cycle

connected disconnected-> not connected componects

sparse dense-> edges are more than vertice

weighted and unweighted: Edges with costs (e.g., flight prices) vs. equal/no costs.

Graph Representation:

Adjacency Matrix :

A B C

A 0 1 1

B 1 0 0

C 1 0 0

Space : $O(V^2)$

for dense this one is better

Adjacency list:

A $\rightarrow$ B, C

B $\rightarrow$ A

C $\rightarrow$ A

space $\rightarrow O(V+E)$

why (no of vertices + how many edges to that)

for sparse graph its better

BFS vs DFS :-

Breadth First Search (BFS) explores nodes level by level using a queue, while Depth First Search (DFS) explores one path deeply using a stack or recursion, then backtracks.

Understanding when to use each approach is a key skill in algorithm design and technical interviews.

◆ Use BFS when

- Finding the shortest path in an unweighted graph
- Problems involving minimum number of steps
- Level-order traversal in trees
- Multi-source expansion problems (like spreading processes)

BFS is particularly useful when the solution is close to the starting node.

◆ Use DFS when

- Finding any valid path rather than the shortest
- Exploring all possibilities (backtracking problems)

- Cycle detection in graphs
- Topological sorting
- Problems where solutions may lie deep in the graph

◆ Key Trade-offs

Memory: -

- BFS uses more memory because it stores entire levels in a queue
- DFS uses less memory since it tracks only the current path

Shortest Path:-

- BFS guarantees the shortest path in unweighted graphs
- DFS does not guarantee shortest paths

Exploration Strategy:-

- BFS works best for shallow solutions
- DFS works better when solutions are deep in the graph

BFS are preferred for grid traversal

DFS

```

      A
    B  C
  D E   F

```

A B D E C F

AS we can see LAST IN FIRST OUT so for DFS we can use stack as we need to backtrack after visiting the neighbours
we use recursive stack

BFS

```

      A
    B  C

```

D E F

A B C D E F

AS we can see FIRST IN FIRST OUT so for BFS we can use queue.

using adjacecny list - $O(V+E)$

using matrix -> $O(v^2)$

Problem1 :->

Knights to move

(-2,-1) 2,3	2,5(-2,1)
(-1,-2) (3,2)	3,6 (-1,2)
(4,4)	
(1,-2) 5,2	5,6(1,2)
(2,-1) 6,3	6,5(2,1)

00	01	02	03	04	05	06	07	08
10	11	12	13	14	15	16	17	18
20	21	22	23	24	25	26	27	28
30	31	32	33	34	35	36	37	38
40	41	42	43	44	45	46	47	48
50	51	52	53	54	55	56	57	58
60	61	62	63	64	65	66	67	68
70	71	72	73	74	75	76	77	78

-1,-2

-2,-1

1,2

2,1

-1,2

2,-1

1,-2

-2,1

TC : $O(n*n)$

problem2:->Reorder Routesm

$n = 6$

connections = $[[0,1],[1,3],[2,3],[4,0],[4,5]]$

correct dir cost = +1 , reverse +0

as we are going outward dirction from city so

0 -> 1 +1

1 -> 0 0

1 -> 3 +1

3 -> 1 0

2 -> 3 +1

3 -> 2 0

4 -> 0 +1

0 -> 4 0

4 -> 5 +1

5 -> 4 0

0 -> (1 +1) , (4,0)

1 -> (0,0),(3,+1)

2-> (3,+1)

3->(1 , 0),(2 ,0)

4->(0 +1),(5 +1)

5->(4 ,0)

$$\begin{aligned}
 \text{cost} &= 1 + (f(1)) \\
 &\quad 1 + f(3) \\
 &\quad \quad 0 + f(2)) + f(4) \\
 &\quad \quad \quad 1 + f(5) \\
 &= 3
 \end{aligned}$$

problem 3 -> cycle in undirected graph

1-2-3-4-1

here dir will be 1→2 and 2→1 so if u can see if node.adj!=
node.parent
can be solved using bfs and dfs

problem 4 -> cycle in directed graph

1->2->3->4->5->1

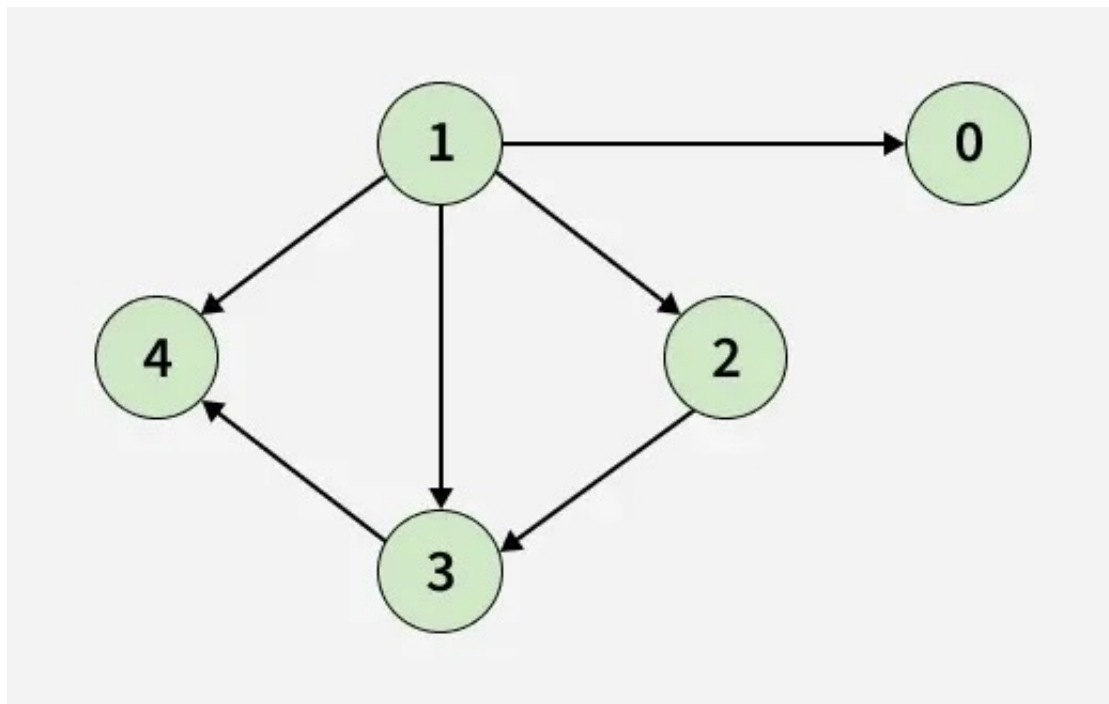
2->4->5->1

if any node is coming twice in the same path which is starting from 1,
1 is coming twice loop is there.

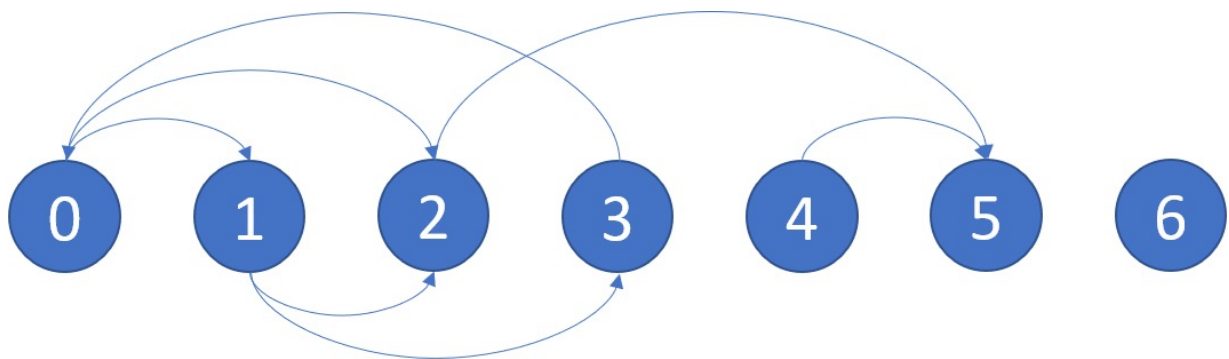
problem 5-> Eventual Safe States

<https://www.geeksforgeeks.org/problems/eventual-safe-states/1>

<https://leetcode.com/problems/find-eventual-safe-states/description/>



here 0 is terminal node and 4 is terminal node
 1 is safe node 3,2 is also



here 5 and 6 is terminal node as there is no outgoing edges
 0 -> 2 -> 5
 0 -> 1 -> 3 -> 1 so 0 is not safe node as for safe node every path from it
 should end at TN
 1 -> 3 -> 0 -> 1 1 is also not a safe node
 2 -> 5 , 2 is also safe node
 4 -> 5, 4 is also a safe node
 so 2,4,5,6

conclusion:->

1. so there should be no loop for safe node
2. we can start from the outgoing node and can store the reverse graph like

5 ->4, 5->2

2->0

so path can never end if its in loop if its ending on any node then for sure that node is a terminal node.

its problem of cycle detection

```
graph = [[1,2],[2,3],[5],[0],[5],[],[[]]]
```

problem 6 -> longestCycle

maintain a map for processing any path and if any node is revisited again just check the last step and current step difference

Topo sort if $u \rightarrow v \rightarrow w$

u should come before v

its only valid for directed acyclic graph

dfs approach use stack for storing the values

Kahns approach

indegree way

shortestPath

1 single source shortest path -

1.1 weighted

+ve weight -> Dijkstra algo

- ve weight -> bellman ford

1.2 unweighted -> BFS is helpful for shortest path

2 multi source shortest path all sources we will check -> Floyd warshall algo

snakes and ladder

```

1 class Solution {
2 public:
3     int snakesAndLadders(vector<vector<int>>> board) {
4         int n = board.size();
5         // #1 making the board easier to comprehend
6         int node = 1;
7         bool flag = true;
8         vector<int> connection(n*n+1, -1);
9         for (int i = n - 1; i >= 0; i--) {
10             if (flag) {
11                 for (int j = 0; j < n; j++) {
12                     if (board[i][j] != -1) {
13                         int destination = board[i][j];
14                         connection[node] = destination;
15                     }
16                     node += 1;
17                 }
18             } else {
19                 for (int j = n - 1; j >= 0; j--) {
20                     if (board[i][j] != -1) {
21                         int destination = board[i][j];
22                         connection[node] = destination;
23                     }
24                     node += 1;
25                 }
26             }
27             flag = !flag;
28         }
29     }
30     // #2 building the graph
31     unordered_map<int, vector<int>> graph;
32     for (int i = 1; i <= n * n - 1; i++) {
33         for (int count = 1; count <= 6; count++) {
34             int nbr = i + count;
35             if (nbr <= n * n) {
36                 if (connection[nbr] != -1) {
37                     // has ladder or snake
38                     graph[i].push_back(connection[nbr]);
39                 } else {
40                     graph[i].push_back(nbr);
41                 }
42             }
43         }
44     }
45 }
46
47 // finding the shortest path
48 int level = 0;
49 vector<int> visited(n * n + 1); // for tcs generated like {{1,1,-1},{1,1,1},{-1,1,1}}
50 queue<int> q;
51 q.push(1);
52 visited[1] = 1;
53 while (!q.empty()) {
54     int sz = q.size();
55     while (sz--) {
56         int pos = q.front();
57         q.pop();
58         if (pos == n * n)
59             return level;
60         for (int nbr : graph[pos]) {
61             if (!visited[nbr]) {
62                 visited[nbr] = 1;
63                 q.push(nbr);
64             }
65         }
66         level++;
67     }
68 }
69 return -1;
70 }
71 };

```

Handwritten notes on the right side of the code editor:

- A circle containing n^2 .
- A box containing $O(n^2)$ and $O(n^2)$ with arrows pointing to them from the circle.
- A box containing vis , cb , and $graph$ with arrows pointing to them from the box.

Dijkstra algo

for weighted graph

bfs cant be used as it tries to find min edges not the min weight
so for that we try to use priorityQueue which always pick the
neighbour with min wight

TC : $O((V + E) \log V)$ using a min heap.